AI Blog Generator: YouTube Video to Blog Article using Django

Oct 27, 2026 · @deven vaghela

Submission Details

Field	Details
Project title	AI Blog Generator: YouTube Video to Blog Article using Django
Subject	Programming With Python
Name	Vaghela Deven Maheshbhai
Enrolment no.	12402080601027
Class	IT A
Batch	1B8
Semester	5th
Submitted to	Nayan Mali
Source code	github.com/devenvaghela57-cloud/ai-blog-generator

Abstract

AI Blog Generator is a Django web application that converts a YouTube video into a written blog article. The user pastes a video link, the backend downloads the audio, a speech-to-text service produces a transcript, and a large language model rewrites it as a proper blog post whose tone follows the video title. Every generated article is saved in a PostgreSQL database against the logged-in user, so it can be opened again later.

The project uses Python 3.13 with Django for the backend, yt-dlp for downloading audio, AssemblyAI for transcription, Google Gemini 2.5 Flash for writing the article, and HTML, Tailwind CSS and JavaScript for the interface.

Introduction

Video is now one of the most common ways to share knowledge, but a video cannot be skimmed, searched or quoted the way text can. Turning a video into an article normally means watching it fully, taking notes and writing everything by hand, which takes a lot of time.

This project automates that work. A user only supplies a YouTube link, and the system returns a readable blog article within a short time. The application also has user accounts, so each person sees only the articles they generated.

Problem statement

To build a web application in Python that takes a YouTube video link as input and automatically produces a well-structured blog article from the spoken content of the video, and stores it for later reading.

Objectives

- Build a working Django web application using Python.
- Let users sign up, log in and log out securely.
- Convert a YouTube link into a blog article automatically.
- Adjust the tone of the article to match the video title.
- Store each article in a PostgreSQL database and show it only to its owner.
- Practise using third-party libraries and APIs from Python.

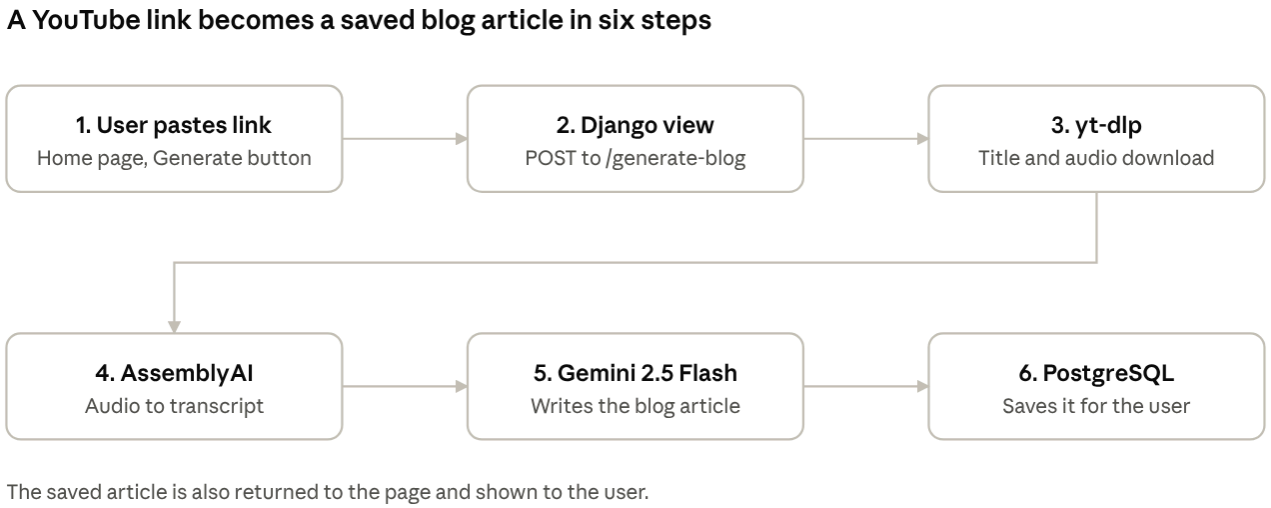
Technology Stack

Layer	Technology	Role in the project
Language	Python 3.13	Backend logic and all integrations
Web framework	Django	Routing, views, templates, authentication, ORM
Database	PostgreSQL	Stores users and generated blog posts
Audio download	yt-dlp	Fetches the video title and downloads the audio
Speech-to-text	AssemblyAI	Converts the audio into a transcript

Layer	Technology	Role in the project
Text generation	Google Gemini 2.5 Flash	Writes the blog article from the transcript
Frontend	HTML, Tailwind CSS, JavaScript	Pages, styling and the call to the backend
Tools	VS Code, uv, pgAdmin, Git and GitHub	Development, packages, database management, version control

System Workflow and Architecture

The application follows a simple pipeline. Everything from the click of the Generate button to the saved article happens in one request handled by the Django backend, which calls three outside services in turn and then writes the result to PostgreSQL.



request workflow · 6 steps

Read the diagram from the top row left to right, then continue on the bottom row. Steps 3 to 5 rely on third-party tools (yt-dlp, AssemblyAI and Gemini); steps 2 and 6 are handled by the Django application itself.

Database Design

The project uses two tables: Django's built-in `User` table for accounts, and a `BlogPost` table that stores every generated article. Each blog post belongs to exactly one user through a foreign key, so one user can have many blog posts.

`BlogPost` field	Holds
`user`	Foreign key to the user who generated the article
`youtube_title`	Title of the YouTube video
`youtube_link`	Link of the YouTube video
`generated_content`	The blog article written by the AI

PostgreSQL is connected through the `DATABASES` setting in `settings.py`, and the tables are created with Django migrations (`python manage.py migrate`).

Modules and Implementation

The application lives in one Django app, `blog_generator`, and its logic sits in `views.py`.

1. Authentication module

- `user_signup` creates a new account after checking that both passwords match.
- `user_login` verifies the username and password and starts a session; wrong details show an error message.
- `user_logout` ends the session.
- The home page is protected with `@login_required`, so only logged-in users can generate blogs.

2. Blog generation module

The `/generate-blog` route accepts a JSON `POST` request containing the YouTube link and runs these functions in order:

1. `yt_title(link)` uses yt-dlp to read the video title without downloading the video.
2. `download_audio(link)` uses yt-dlp to download the best audio stream into the `media/` folder.
3. `get_transcription(link)` sends that audio file to AssemblyAI and returns the transcript text.
4. `generate_blog_from_transcription(transcription, title)` sends the transcript and the title to Gemini 2.5 Flash. The prompt asks for a proper blog article, not a video summary, with the tone matched to the title.
5. The article is saved as a `BlogPost` for the logged-in user and returned to the page as JSON.

The title and transcription steps are wrapped in error handling, so a failure there returns a clear JSON error message instead of crashing the server.

3. Saved posts module

- `blog_list` shows all articles generated by the current user.
- `blog_details` shows one article with its video title and link. If the article belongs to another user, the visitor is redirected to the home page.

Pages

URL	Purpose
`/`	Home page: enter a link, see the generated article
`/login` and `/signup`	Authentication pages
`/generate-blog`	Backend endpoint that creates the article
`/blog-list`	List of saved blog posts
`/blog-details/<id>/`	One saved blog post

Project Structure and How to Run

```
final django/
├── backend/
|   └── ai_blog_app/
|       ├── ai_blog_app/        # project settings and URLs
|       ├── blog_generator/     # views, models, URLs, migrations
|       ├── templates/          # HTML templates used by Django
|       └── manage.py
├── front/                  # static HTML pages (frontend design)
├── screenshots/
├── requirements.txt
└── README.md
```

Steps to run the project on a new computer:

1. Clone the repository from GitHub and create a virtual environment.
2. Install the libraries with `uv pip install -r requirements.txt` (or `pip install -r requirements.txt`).
3. Create a PostgreSQL database named `db`.

4. Replace the placeholders `YOUR_ASSEMBLYAI_API_KEY`, `YOUR_GEMINI_API_KEY` and `YOUR_DATABASE_PASSWORD` with real values.
5. Run `python manage.py migrate`, then `python manage.py runserver`.
6. Open `http://127.0.0.1:8000` in the browser.

Challenges Faced and Solutions

Challenge	Solution
The `pytubefix` library was blocked by YouTube with a bot-detection error, so the title and audio could not be fetched.	Replaced it with `yt-dlp`, which is updated often to keep up with YouTube changes.
Renaming the downloaded audio file to `.mp3` failed on Windows when a file with that name already existed.	Removed the renaming step. yt-dlp saves the file under the video id, and AssemblyAI accepts that format directly.
API keys and the database password must not reach a public GitHub repository.	Replaced them with placeholders such as `YOUR_GEMINI_API_KEY` before committing, and used a `.gitignore` to keep the virtual environment, `media/` folder, local database files and `.env` out of the repository.
A failing external service could crash the request.	Wrapped the title and transcription steps in error handling that returns a clear JSON error, and the Gemini call in its own try/except block that logs the error.

Limitations and Future Scope

Current limitations

- The app depends on three outside services (YouTube, AssemblyAI and Gemini), so it stops working if any of them changes or reaches its usage limit.
- Long videos take more time to download and transcribe. If the Gemini call fails, the error message it returns is saved in place of the article.
- The generated text is shown as plain text, so the Markdown symbols from the AI (such as `##` and `**`) appear as they are.
- API keys are written in the code instead of being read from environment variables.

Future scope

- Load API keys and database settings from environment variables.
- Render the Markdown output as proper headings and bold text.
- Allow editing and deleting saved blog posts.
- Export articles as PDF or Markdown files.
- Let the user choose the output language and writing style.
- Add CSRF token handling to the generate request and deploy the project with Docker.

Conclusion

AI Blog Generator shows how a Python and Django application can combine several services into one useful tool. A YouTube link goes in, and a saved, readable blog article comes out. The project covered user authentication, database design with PostgreSQL, working with external APIs, error handling and version control with GitHub.

The most important lesson was handling real-world changes: when YouTube blocked the original download library, switching to yt-dlp restored the project. With the improvements listed above, the application can become a more complete and secure content tool.

References

- Django documentation
- PostgreSQL documentation
- yt-dlp on GitHub
- AssemblyAI documentation
- Google Gemini API documentation
- Tailwind CSS documentation
- freeCodeCamp.org: Python backend development course with Django (Code With Tim), on YouTube
- Project source code on GitHub